

# End-to-End Kubernetes on AWS Lab: Deploying and Validating an Application on Amazon EKS

Hands-on Lab Guide

July 2026

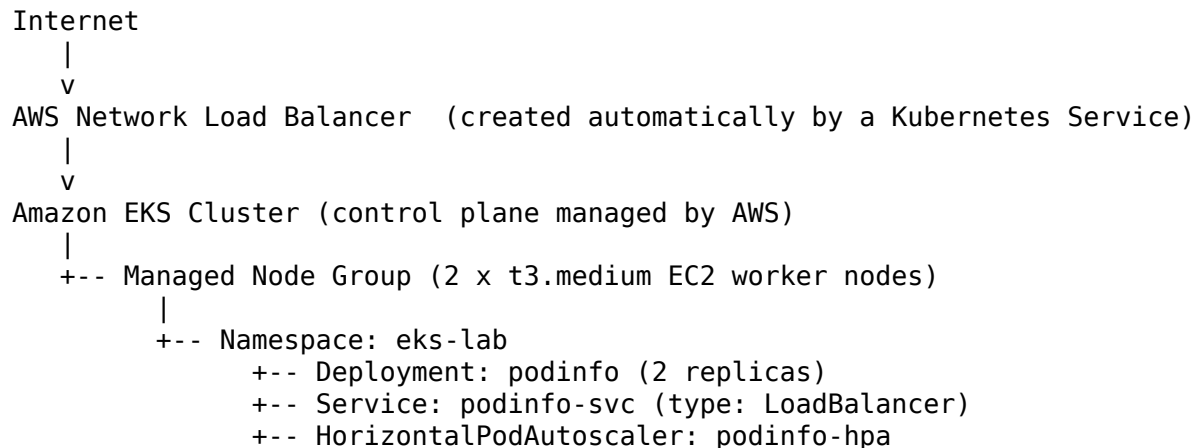
## Contents

|                                                                                      |           |
|--------------------------------------------------------------------------------------|-----------|
| <b>Lab Overview</b>                                                                  | <b>1</b>  |
| What you will build . . . . .                                                        | 2         |
| Learning objectives . . . . .                                                        | 2         |
| Estimated time and cost . . . . .                                                    | 2         |
| Lab files provided . . . . .                                                         | 2         |
| <b>Prerequisites</b>                                                                 | <b>3</b>  |
| <b>Step 1: Install the required CLI tools</b>                                        | <b>3</b>  |
| <b>Step 2: Configure AWS credentials</b>                                             | <b>4</b>  |
| <b>Step 3: Review and create the EKS cluster</b>                                     | <b>4</b>  |
| <b>Step 4: Deploy the sample application</b>                                         | <b>6</b>  |
| 4.1 Namespace . . . . .                                                              | 6         |
| 4.2 Deployment . . . . .                                                             | 6         |
| 4.3 Service (exposes the app via an AWS Load Balancer) . . . . .                     | 7         |
| 4.4 HorizontalPodAutoscaler . . . . .                                                | 7         |
| 4.5 Deploy everything . . . . .                                                      | 8         |
| <b>Step 5: Validate the deployment end-to-end</b>                                    | <b>8</b>  |
| <b>Step 6: Watch the application autoscale under load (optional but recommended)</b> | <b>10</b> |
| <b>Step 7: Simulate a rolling update (optional)</b>                                  | <b>10</b> |
| <b>Step 8: Clean up all AWS resources</b>                                            | <b>10</b> |
| <b>Troubleshooting</b>                                                               | <b>11</b> |
| <b>Summary</b>                                                                       | <b>12</b> |

## Lab Overview

In this lab you will provision a real **Amazon EKS (Elastic Kubernetes Service)** cluster on AWS, deploy a containerized application onto it, expose it to the internet through an AWS Load Balancer, validate that it is running correctly, watch it auto-scale under load, and then tear everything down cleanly.

## What you will build



## Learning objectives

By the end of this lab you will be able to:

1. Install and configure the AWS CLI, kubectl, eksctl, and helm.
2. Provision an EKS cluster and a managed node group using an eksctl YAML spec.
3. Deploy a sample application to the cluster using Kubernetes manifests.
4. Expose the application to the internet using a Kubernetes Service of type LoadBalancer.
5. Validate the deployment end-to-end (node health, pod health, logs, connectivity).
6. Configure and observe a Horizontal Pod Autoscaler (HPA) scaling pods under load.
7. Clean up all AWS resources to avoid ongoing charges.

## Estimated time and cost

- **Time:** 60-90 minutes (cluster creation alone takes 15-20 minutes).
- **Cost:** EKS control plane costs approximately \$0.10/hour. Two t3.medium nodes and a Network Load Balancer add a small hourly cost. **Always run the cleanup step (Step 8) when you are done** to avoid ongoing charges.

## Lab files provided

This lab ships with the following files, referenced throughout the steps below:

| File                         | Purpose                                         |
|------------------------------|-------------------------------------------------|
| manifests/eks-cluster.yaml   | eksctl cluster specification                    |
| manifests/00-namespace.yaml  | Kubernetes namespace for the lab                |
| manifests/01-deployment.yaml | Application Deployment (podinfo, 2 replicas)    |
| manifests/02-service.yaml    | LoadBalancer Service exposing the app           |
| manifests/03-hpa.yaml        | HorizontalPodAutoscaler for the app             |
| scripts/01-install-tools.sh  | Installs AWS CLI, kubectl, eksctl, helm (Linux) |
| scripts/02-create-cluster.sh | Creates the EKS cluster and updates kubeconfig  |
| scripts/03-deploy-app.sh     | Installs metrics-server and deploys the app     |
| scripts/04-validate.sh       | Runs all validation checks automatically        |
| scripts/05-load-test.sh      | Generates load to trigger autoscaling           |
| scripts/06-cleanup.sh        | Deletes the app and the EKS cluster             |

---

## Prerequisites

Before starting, make sure you have:

- An **AWS account** with permissions to create EKS clusters, EC2 instances, VPCs, IAM roles, and Elastic Load Balancers (AdministratorAccess is simplest for a lab environment).
  - An **IAM user or role with programmatic access** (Access Key ID + Secret Access Key), or AWS SSO configured.
  - A Linux, macOS, or WSL2 terminal. (Windows users: run everything inside WSL2 or Git Bash.)
  - Roughly 90 minutes of uninterrupted time, since cluster creation cannot be paused.
- 

## Step 1: Install the required CLI tools

You need four tools: **AWS CLI v2**, **kubectl**, **eksctl**, and **helm**.

Run the provided script:

```
cd eks-lab/scripts
bash 01-install-tools.sh
```

This script performs the following installs (shown here so you understand what it does):

```
# AWS CLI v2
curl -s "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip -q -o awscliv2.zip
sudo ./aws/install --update

# kubectl (latest stable)
KVER=$(curl -Ls https://dl.k8s.io/release/stable.txt)
curl -LO "https://dl.k8s.io/release/${KVER}/bin/linux/amd64/kubectl"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/

# eksctl
curl -sL "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Linux_amd64."
sudo mv /tmp/eksctl /usr/local/bin/

# helm
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod +x get_helm.sh && ./get_helm.sh
```

**macOS users** can instead use Homebrew:

```
brew install awscli kubectl eksctl helm
```

## Verify installation

```
aws --version
kubectl version --client
```

```
eksctl version
helm version
```

You should see a version number printed for each tool with no errors.

---

## Step 2: Configure AWS credentials

Configure the AWS CLI with your credentials:

```
aws configure
```

You will be prompted for:

```
AWS Access Key ID [None]: <your access key>
AWS Secret Access Key [None]: <your secret key>
Default region name [None]: us-east-1
Default output format [None]: json
```

Confirm your identity is recognized correctly:

```
aws sts get-caller-identity
```

Expected output looks like:

```
{
  "UserId": "AIDAEXAMPLE123456",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/your-username"
}
```

If this fails, double check your access key/secret and that your IAM identity has sufficient permissions.

---

## Step 3: Review and create the EKS cluster

The cluster topology is defined declaratively in `manifests/eks-cluster.yaml`:

```
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: eks-lab-cluster
  region: us-east-1
  version: "1.30"

availabilityZones:
- us-east-1a
- us-east-1b

managedNodeGroups:
- name: eks-lab-nodegroup
  instanceType: t3.medium
```

```

minSize: 2
maxSize: 4
desiredCapacity: 2
volumeSize: 20
ssh:
  allow: false
labels:
  role: worker
tags:
  environment: lab
  project: eks-lab

iam:
  withOIDC: true

cloudWatch:
  clusterLogging:
    enableTypes: ["api", "audit", "authenticator"]

```

Key points:

- `withOIDC: true` provisions an IAM OIDC identity provider for the cluster, which is required later for things like the metrics-server, cluster-autoscaler, or the AWS Load Balancer Controller to assume fine-grained IAM roles (IRSA).
- `managedNodeGroups` tells EKS to manage the EC2 worker nodes for you (patching, scaling, lifecycle) instead of you managing raw Auto Scaling Groups.
- Edit the `region` field if you want a different AWS region.

## Create the cluster

```

cd eks-lab/scripts
bash 02-create-cluster.sh

```

Or run the equivalent commands manually:

```

eksctl create cluster -f ../manifests/eks-cluster.yaml
aws eks update-kubeconfig --name eks-lab-cluster --region us-east-1

```

This provisions, in order: a VPC with public/private subnets, the EKS control plane, an IAM OIDC provider, and a managed node group of 2 EC2 instances. **This step takes 15-20 minutes.** `eksctl` prints progress as it goes.

## Verify the cluster and nodes

```

kubectl get nodes -o wide

```

Expected output (2 nodes in Ready state):

| NAME                           | STATUS | ROLES  | AGE | VERSION     |
|--------------------------------|--------|--------|-----|-------------|
| ip-192-168-45-112.ec2.internal | Ready  | <none> | 3m  | v1.30.x-eks |
| ip-192-168-78-201.ec2.internal | Ready  | <none> | 3m  | v1.30.x-eks |

If nodes show `NotReady`, wait another minute and re-check — they are usually still finishing bootstrap.

## Step 4: Deploy the sample application

The application is **podinfo**, a small, purpose-built Go microservice commonly used in Kubernetes demos. It exposes a web UI, a REST API, and `/healthz` / `/readyz` endpoints, which makes it ideal for demonstrating health checks, rolling updates, and load balancing.

### 4.1 Namespace

`manifests/00-namespace.yaml`

```
apiVersion: v1
kind: Namespace
metadata:
  name: eks-lab
  labels:
    purpose: eks-end-to-end-lab
```

### 4.2 Deployment

`manifests/01-deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: podinfo
  namespace: eks-lab
  labels:
    app: podinfo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: podinfo
  template:
    metadata:
      labels:
        app: podinfo
    spec:
      containers:
        - name: podinfo
          image: ghcr.io/stefanprodan/podinfo:6.7.1
          ports:
            - containerPort: 9898
              name: http
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 250m
              memory: 128Mi
          readinessProbe:
            httpGet:
              path: /readyz
```

```

    port: 9898
    initialDelaySeconds: 5
    periodSeconds: 5
  livenessProbe:
    httpGet:
      path: /healthz
      port: 9898
    initialDelaySeconds: 10
    periodSeconds: 10

```

Notes: - resources.requests are what the Kubernetes scheduler uses to place pods; they are also what the HPA measures utilization against. - readinessProbe / livenessProbe let Kubernetes automatically detect and recover from application failures.

### 4.3 Service (exposes the app via an AWS Load Balancer)

manifests/02-service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: podinfo-svc
  namespace: eks-lab
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
spec:
  type: LoadBalancer
  selector:
    app: podinfo
  ports:
    - name: http
      port: 80
      targetPort: 9898
      protocol: TCP

```

Setting type: LoadBalancer tells the in-cluster AWS cloud-controller to automatically provision a real **Network Load Balancer (NLB)** in your AWS account and point it at the pods.

### 4.4 HorizontalPodAutoscaler

manifests/03-hpa.yaml

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: podinfo-hpa
  namespace: eks-lab
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: podinfo
  minReplicas: 2
  maxReplicas: 6

```

```
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 50
```

This tells Kubernetes to scale the podinfo deployment between 2 and 6 replicas, adding pods whenever average CPU utilization exceeds 50% of the requested 100m.

## 4.5 Deploy everything

The HPA requires **metrics-server** to be running in the cluster (it is not installed by default on EKS), so the deploy script installs that first:

```
cd eks-lab/scripts
bash 03-deploy-app.sh
```

Equivalent manual commands:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download
kubectl -n kube-system rollout status deployment/metrics-server --timeout=120s

kubectl apply -f ../manifests/00-namespace.yaml
kubectl apply -f ../manifests/01-deployment.yaml
kubectl apply -f ../manifests/02-service.yaml
kubectl apply -f ../manifests/03-hpa.yaml

kubectl -n eks-lab rollout status deployment/podinfo --timeout=180s
```

---

## Step 5: Validate the deployment end-to-end

Run the automated validation script:

```
cd eks-lab/scripts
bash 04-validate.sh
```

This script performs the checks below. You can also run them manually one at a time to understand each layer of the stack.

### 5.1 Are the worker nodes healthy?

```
kubectl get nodes -o wide
```

Look for STATUS = Ready on both nodes.

### 5.2 Are the pods running?

```
kubectl -n eks-lab get pods -o wide
```

Expected:

| NAME                     | READY | STATUS  | RESTARTS | AGE |
|--------------------------|-------|---------|----------|-----|
| podinfo-7d4f9c9c8f-2xk9p | 1/1   | Running | 0        | 2m  |
| podinfo-7d4f9c9c8f-9jd3q | 1/1   | Running | 0        | 2m  |

READY = 1/1 confirms the readiness probe is passing.

### 5.3 Inspect pod details and recent events

```
kubectl -n eks-lab describe pod <pod-name>
```

Check the Events section at the bottom for Scheduled, Pulled, Created, Started — with no Warning/Failed events.

### 5.4 Check application logs

```
kubectl -n eks-lab logs <pod-name>
```

You should see podinfo's startup log lines, including the HTTP server binding to port 9898.

### 5.5 Confirm the Service and Load Balancer

```
kubectl -n eks-lab get svc podinfo-svc
```

Expected (an AWS-generated hostname appears under EXTERNAL-IP once the NLB finishes provisioning, typically 2-3 minutes):

| NAME        | TYPE         | CLUSTER-IP   | EXTERNAL-IP                                         | PORT(S)      |
|-------------|--------------|--------------|-----------------------------------------------------|--------------|
| podinfo-svc | LoadBalancer | 10.100.32.55 | a1b2c3d4e5f6-1234567890.elb.us-east-1.amazonaws.com | 80:31820/TCP |

### 5.6 Test connectivity through the public Load Balancer

```
curl http://<EXTERNAL-IP-from-above>
```

Expected: a JSON response from podinfo containing its hostname, version, and runtime info — proof that traffic is flowing Internet -> NLB -> Service -> Pod.

### 5.7 (Optional) Test connectivity without waiting for the Load Balancer

While the NLB is provisioning, you can validate connectivity immediately using port-forward:

```
kubectl -n eks-lab port-forward svc/podinfo-svc 8080:80
# in another terminal:
curl http://localhost:8080
```

### 5.8 Confirm the HPA is tracking metrics

```
kubectl -n eks-lab get hpa
```

Expected:

| NAME        | REFERENCE          | TARGETS | MINPODS | MAXPODS | REPLICAS |
|-------------|--------------------|---------|---------|---------|----------|
| podinfo-hpa | Deployment/podinfo | 12%/50% | 2       | 6       | 2        |

A TARGETS value other than <unknown> confirms metrics-server is feeding real CPU data to the HPA.

**Your application is fully validated once all 8 checks above pass.**

---

## Step 6: Watch the application autoscale under load (optional but recommended)

Open a second terminal and watch the HPA live:

```
kubectl -n eks-lab get hpa -w
```

In your first terminal, generate load against the service from inside the cluster:

```
cd eks-lab/scripts
bash 05-load-test.sh
```

This runs a temporary busybox pod in a tight loop hitting the service. Within 1-3 minutes you should see TARGETS climb above 50% and REPLICAS increase from 2 toward 6. Stop the load test with Ctrl+C — replicas will scale back down to 2 automatically after a cooldown period (default 5 minutes).

---

## Step 7: Simulate a rolling update (optional)

This demonstrates zero-downtime deployments, a core Kubernetes capability:

```
kubectl -n eks-lab set image deployment/podinfo podinfo=ghcr.io/stefanprodan/podinfo:6.7.0
kubectl -n eks-lab rollout status deployment/podinfo
```

Watch pods being replaced one at a time with no downtime:

```
kubectl -n eks-lab get pods -w
```

Roll back if needed:

```
kubectl -n eks-lab rollout undo deployment/podinfo
```

---

## Step 8: Clean up all AWS resources

**This step is mandatory** to avoid ongoing AWS charges. Deleting the Kubernetes Service first ensures the AWS Load Balancer is de-provisioned cleanly before the cluster itself is deleted.

```
cd eks-lab/scripts
bash 06-cleanup.sh
```

Equivalent manual commands:

```
kubectl delete -f ../manifests/03-hpa.yaml --ignore-not-found
kubectl delete -f ../manifests/02-service.yaml --ignore-not-found
kubectl delete -f ../manifests/01-deployment.yaml --ignore-not-found
```

```
kubectl delete -f ../manifests/00-namespace.yaml --ignore-not-found
```

```
sleep 30 # let the Load Balancer/ENIs fully de-provision
```

```
eksctl delete cluster --name eks-lab-cluster --region us-east-1
```

## Final verification

After the script finishes (10-15 minutes), confirm in the **AWS Console** that the following no longer exist:

- EKS cluster eks-lab-cluster
- EC2 instances tagged project=eks-lab
- Any Load Balancer named with the podinfo-svc service UID
- The VPC created for the cluster (eksctl deletes this automatically)

You can also confirm via CLI:

```
aws eks list-clusters
```

```
aws ec2 describe-instances --filters "Name=tag:project,Values=eks-lab" --query "Reservations"
```

Both commands should return empty results.

## Troubleshooting

| Symptom                                                | Likely cause                                                                | Fix                                                                                              |
|--------------------------------------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| eksctl create cluster fails with IAM permission errors | Your IAM user lacks EKS/EC2/IAM permissions                                 | Attach AdministratorAccess (lab only) or the specific EKS/EC2/IAM policies                       |
| Nodes stuck in NotReady                                | Networking/security group issue, or still bootstrapping                     | Wait 2-3 minutes; check kubectl describe node <name> for errors                                  |
| Pods stuck in Pending                                  | Insufficient CPU/memory on nodes, or no nodes yet                           | kubectl describe pod <name> to see scheduling events; check node capacity with kubectl top nodes |
| EXTERNAL-IP stays <pending>                            | NLB still provisioning, or missing IAM permissions for the cloud controller | Wait 2-3 minutes; check kubectl -n eks-lab describe svc podinfo-svc for events                   |
| HPA shows <unknown> under TARGETS                      | metrics-server not running or not ready                                     | kubectl -n kube-system get pods -l k8s-app=metrics-server; re-run step 4.5                       |
| curl to the Load Balancer times out                    | Security group rules, or targets not yet healthy                            | Wait 1-2 minutes for NLB target health checks to pass; re-run 04-validate.sh                     |

## Summary

You have provisioned a managed Kubernetes cluster on AWS with EKS, deployed a containerized application with health checks, exposed it to the public internet through a native AWS Load Balancer, validated every layer of the stack from node to HTTP response, observed automatic horizontal scaling under load, performed a zero-downtime rolling update, and cleanly torn down all billable resources.

This same pattern — `eksctl` cluster spec + namespaced manifests + Service + HPA — is the foundation for running production workloads on EKS.